Agentic ~~Vibe~~ Coding of Translators

FP Seminar, CSE Gothenburg, 16 October 2026

Aarne Ranta

Department of Computer Science and Engineering
Chalmers University of Technology and University of Gothenburg

aarne.ranta@cse.gu.se

Informath

NextReason

MALINCA

European Research Council
Established by the European Commission

Context

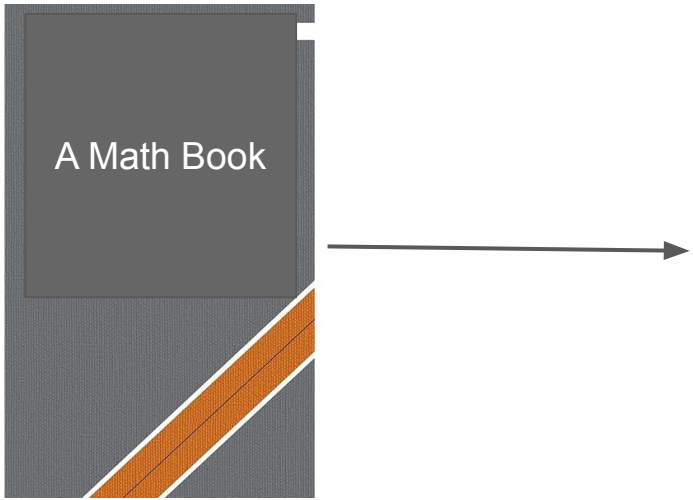
NAVIER–STOKES LOST IN TRANSLATION
–
WHY LEAN VERIFICATION OF AI AUTOFORMALISATION DOES NOT GUARANTEE CORRECT NATURAL LANGUAGE PROOFS

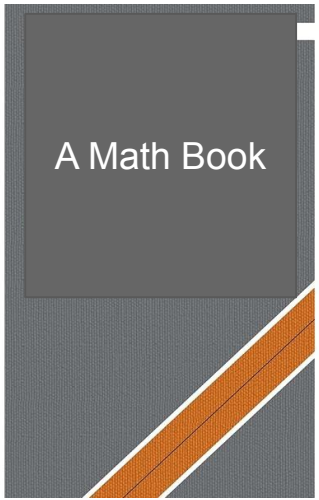
A. BASTOUNIS, F. CIRCELLI, AND A. C. HANSEN*

6 Oct 2026
https://arxiv.org/abs/2610.08144

(b) The model, or some other model, should be prompted to produce a version of each proof that is written up in a style that follows the conventions of a traditional mathematical paper. They should contain friendly introductions and precise theorem statements and proofs. They should not be full of wordy reasoning and non-standard terminology that renders them virtually incomprehensible.

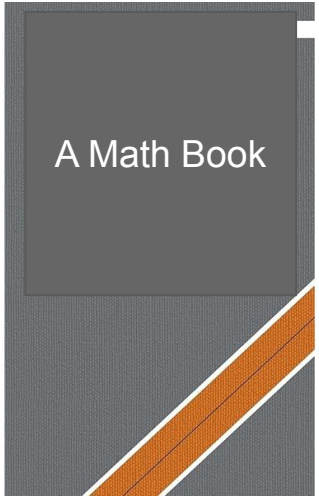
"Responsible Release of AI-Generated Mathematics"
by Advisory Group on Mathematics and Artificial Intelligence
September 29, 2026
https://agmai.org/general-sep29/


A Math Book



LLM autoformalization

```
t0 a (t0 b c)) (t0 (t0 a b) c)) => match_Aop a1
a2 (plus Type2 (nd 1)) (__ : Aop a1 a2 => univ
Type2) (u0 : (__ : a1 -> __1 : a1 -> a1) => u1
: (a : a1 -> Eq (u0 a2 a) a) => u2 : (a : a1 ->
Eq (u0 a a2) a) => u3 : (a : a1 -> b : a1 -> c
: a1 -> Eq (u0 a (u0 b c)) (u0 (u0 a b) c)) =>
prod (plus Type1 (nd 1)) Type1 (univ Type1) (P
: Type1 => prod Type1 Type1 (prod Prop Type1
(Eq (R0 (prod Set Set a1 (__ : a1 => prod Set
Set a1 (__1 : a1 => a1))) t0) u0) (e0 : Eq (R0
(prod Set Set a1 (__ : a1 => prod Set Set a1
(__1 : a1 => a1))) t0) u0 => prod Prop Type1
(Eq (R1 (prod Set Set a1 (__ : a1 => prod Set
Set a1 (__1 : a1 => a1))) t0 (x_19 : (__ : a1
-> __1 : a1 -> a1) => _x_20 : Eq t0 x_19 => (x0
: (__ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0
=> prod Set Prop a1 (a : a1 => Eq (x0 a2 a) a))
x_19 _x_20) t1 u0 e0) u1) (e1 : Eq (R1 (prod
Set Set a1 (__ : a1 => prod Set Set a1 (__1 :
a1 => a1))) t0 (x_19 : (__ : a1 -> __1 : a1 ->
a1) => _x_20 : Eq t0 x_19 => (x0 : (__ : a1 ->
__1 : a1 -> a1) => p0 : Eq t0 x0 => prod Set
Prop a1 (a : a1 => Eq (x0 a2 a) a)) x_19 _x_20)
t1 u0 e0) u1 => prod Prop Type1 (Eq (R2 (prod
Set Set a1 (__ : a1 => prod Set Set a1 (__1 :
a1 => a1))) t0 (x0 : (__ : a1 -> __1 : a1 ->
a1) => __ : Eq t0 x0 => (x01 : (__1 : a1 -> __2
: a1 -> a1) => p0 : Eq t0 x01 => prod Set Prop
a1 (a : a1 => Eq (x01 a2 a) a)) x0 __) t1 (x0 :
(__ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0 =>
x1 : (x01 : (__ : a1 -> __1 : a1 -> a1) => p01
: Eq t0 x01 => prod Set Prop a1 (a : a1 => Eq
(x01 a2 a) a)) x0 p0 => __ : Eq (R1 (prod Set
Set a1 (__ : a1 => prod Set Set a1 (__1 : a1 =>
```



LLM autoformalization

```
t0 a (t0 b c)) (t0 (t0 a b) c)) => match_Aop a1
a2 (plus Type2 (nd 1)) (__ : Aop a1 a2 => univ
Type2) (u0 : (__ : a1 -> __1 : a1 -> a1) => u1
: (a : a1 -> Eq (u0 a2 a) a) => u2 : (a : a1 ->
Eq (u0 a a2) a) => u3 : (a : a1 -> b : a1 -> c
: a1 -> Eq (u0 a (u0 b c)) (u0 (u0 a b) c)) =>
prod (plus Type1 (nd 1)) Type1 (univ Type1) (P
: Type1 => prod Type1 Type1 (prod Prop Type1
(Eq (R0 (prod Set Set a1 (__ : a1 => prod Set
Set a1 (__1 : a1 => a1))) t0) u0) (e0 : Eq (R0
(prod Set Set a1 (__ : a1 => prod Set Set a1
(__1 : a1 => a1))) t0) u0 => prod Prop Type1
(Eq (R1 (prod Set Set a1 (__ : a1 => prod Set
Set a1 (__1 : a1 => a1))) t0 (x_19 : (__ : a1
-> __1 : a1 -> a1) => _x_20 : Eq t0 x_19 => (x0
: (__ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0
=> prod Set Prop a1 (a : a1 => Eq (x0 a2 a) a))
x_19 _x_20) t1 u0 e0) u1) (e1 : Eq (R1 (prod
Set Set a1 (__ : a1 => prod Set Set a1 (__1 :
a1 => a1))) t0 (x_19 : (__ : a1 -> __1 : a1 ->
a1) => _x_20 : Eq t0 x_19 => (x0 : (__ : a1 ->
__1 : a1 -> a1) => p0 : Eq t0 x0 => prod Set
Prop a1 (a : a1 => Eq (x0 a2 a) a)) x_19 _x_20)
t1 u0 e0) u1 => prod Prop Type1 (Eq (R2 (prod
Set Set a1 (__ : a1 => prod Set Set a1 (__1 :
a1 => a1))) t0 (x0 : (__ : a1 -> __1 : a1 ->
a1) => __ : Eq t0 x0 => (x01 : (__1 : a1 -> __2
: a1 -> a1) => p0 : Eq t0 x01 => prod Set Prop
a1 (a : a1 => Eq (x01 a2 a) a)) x0 __) t1 (x0 :
(__ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0 =>
x1 : (x01 : (__ : a1 -> __1 : a1 -> a1) => p01
: Eq t0 x01 => prod Set Prop a1 (a : a1 => Eq
(x01 a2 a) a)) x0 p0 => __ : Eq (R1 (prod Set
Set a1 (__ : a1 => prod Set Set a1 (__1 : a1 =>
```

symbolic informalization

Thm03. $\mathbb{Q}$ is denumerable.

Thm03a.

$$|\mathbb{N}| = |\mathbb{Q}|.$$

Thm04. Let u and v be vectors. Then $u \perp v$ implies $\|u+v\| = \sqrt{\|u\|^2 + \|v\|^2}$.

Thm07. Let $p, q \in \mathbb{N}$. Then if p and q are prime, then

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{\frac{p-1}{2}\frac{q-1}{2}}.$$

Thm09. Let c be a circle. Let r be a real number. Assume that $r = radius(c)$. Then the area of c is equal to $\pi \times r^2$.

Thm10FermatLittle. Let p be a natural number. Assume that p is prime. Let a be an integer. Then there exists an integer q, such that $a^p - a = p \times q$.

Thm11. $p \geq n$ and p is prime for a natural number p for all natural numbers n.

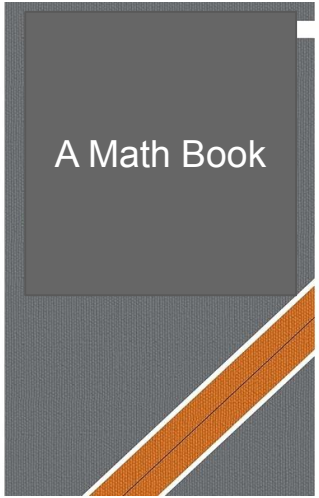
Thm19. $n = a^2 + b^2 + c^2 + d^2$ for some natural numbers a, b, c and d for all natural numbers n.

Thm20a. Let p be a natural number. Assume that p is prime. Let k be a natural number. Assume that $p = 4 \times k + 1$. Then there exists a natural number x, such that there exists a natural number y, such that $p = x^2 + y^2$.

Thm20b. Let p be a natural number. Assume that p is prime. Assume that $p \equiv 1 \bmod 4$. Then there exists a natural number x, such that there exists a natural number y, such that $p = x^2 + y^2$.

Thm22. It is not the case that $\mathbb{R}$ is denumerable.

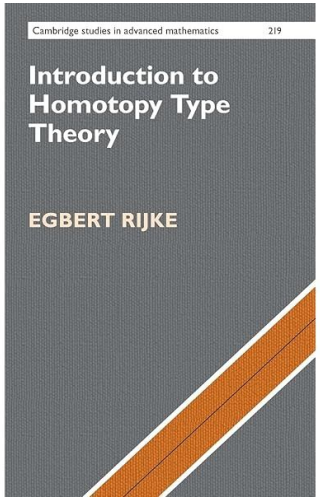
Mock-up example: the code and the text do not correspond to each other.



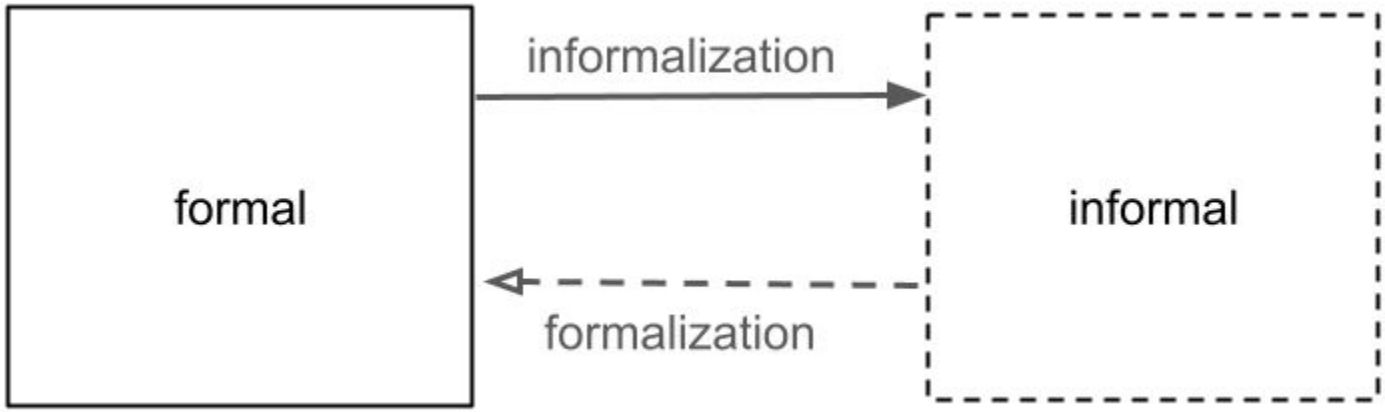
LLM autoformalization

```
t0 a (t0 b c)) (t0 (t0 a b) c)) => match_Aop a1
a2 (plus Type2 (nd 1)) (__ : Aop a1 a2 => univ
Type2) (u0 : (__ : a1 -> __1 : a1 -> a1) => u1
: (a : a1 -> Eq (u0 a2 a) a) => u2 : (a : a1 ->
Eq (u0 a a2) a) => u3 : (a : a1 -> b : a1 -> c
: a1 -> Eq (u0 a (u0 b c)) (u0 (u0 a b) c)) =>
prod (plus Type1 (nd 1)) Type1 (univ Type1) (P
: Type1 => prod Type1 Type1 (prod Prop Type1
(Eq (R0 (prod Set Set a1 (__ : a1 => prod Set
Set a1 (__1 : a1 => a1))) t0) u0) (e0 : Eq (R0
(prod Set Set a1 (__ : a1 => prod Set Set a1
(__1 : a1 => a1))) t0) u0 => prod Prop Type1
(Eq (R1 (prod Set Set a1 (__ : a1 => prod Set
Set a1 (__1 : a1 => a1))) t0 (x_19 : (__ : a1
-> __1 : a1 -> a1) => _x_20 : Eq t0 x_19 => (x0
: (__ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0
=> prod Set Prop a1 (a : a1 => Eq (x0 a2 a) a))
x_19 _x_20) t1 u0 e0) u1) (e1 : Eq (R1 (prod
Set Set a1 (__ : a1 => prod Set Set a1 (__1 :
a1 => a1))) t0 (x_19 : (__ : a1 -> __1 : a1 ->
a1) => _x_20 : Eq t0 x_19 => (x0 : (__ : a1 ->
__1 : a1 -> a1) => p0 : Eq t0 x0 => prod Set
Prop a1 (a : a1 => Eq (x0 a2 a) a)) x_19 _x_20)
t1 u0 e0) u1 => prod Prop Type1 (Eq (R2 (prod
Set Set a1 (__ : a1 => prod Set Set a1 (__1 :
a1 => a1))) t0 (x0 : (__ : a1 -> __1 : a1 ->
a1) => __ : Eq t0 x0 => (x01 : (__1 : a1 -> __2
: a1 -> a1) => p0 : Eq t0 x01 => prod Set Prop
a1 (a : a1 => Eq (x01 a2 a) a)) x0 __) t1 (x0 :
(__ : a1 -> __1 : a1 -> a1) => p0 : Eq t0 x0 =>
x1 : (x01 : (__ : a1 -> __1 : a1 -> a1) => p01
: Eq t0 x01 => prod Set Prop a1 (a : a1 => Eq
(x01 a2 a) a)) x0 p0 => __ : Eq (R1 (prod Set
Set a1 (__ : a1 => prod Set Set a1 (__1 : a1 =>
```

symbolic informalization

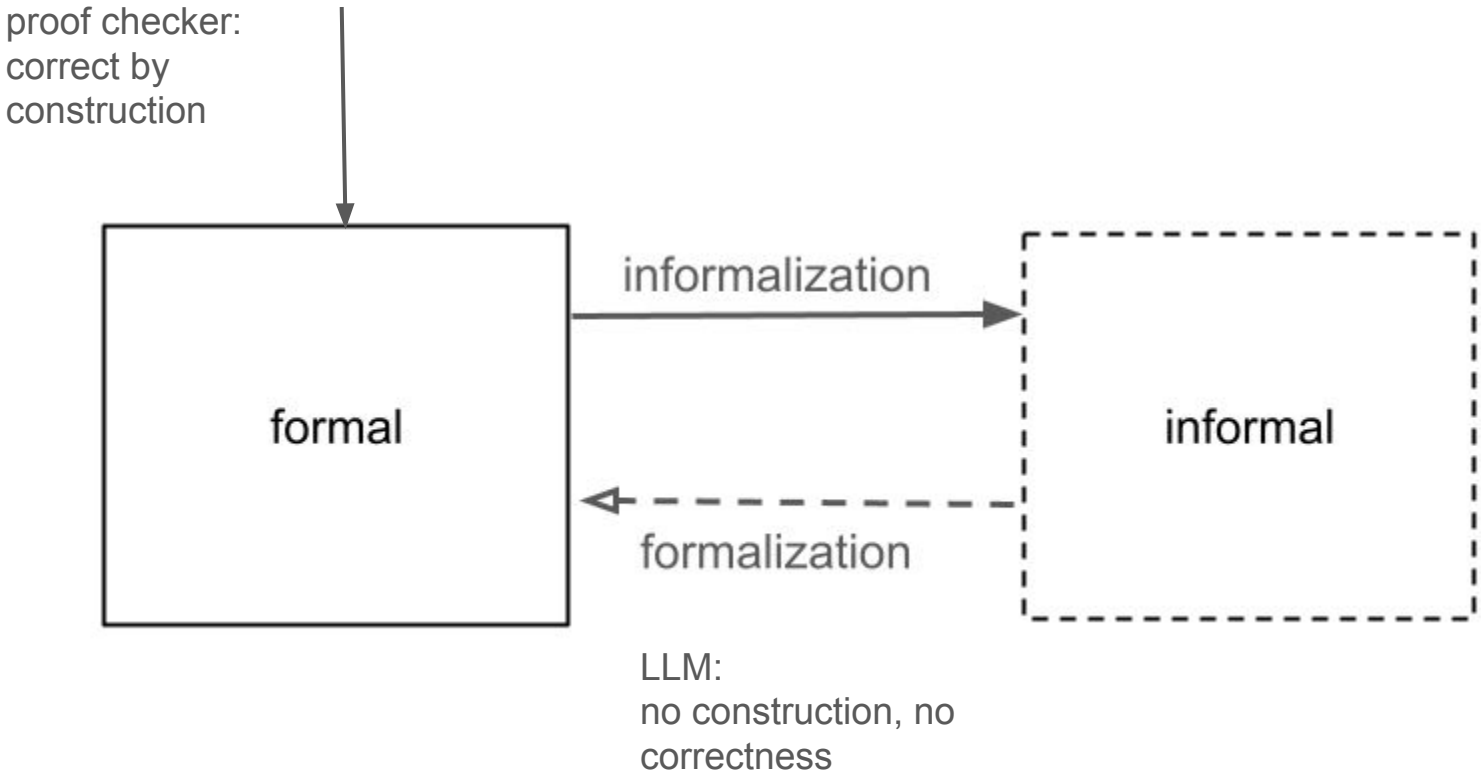


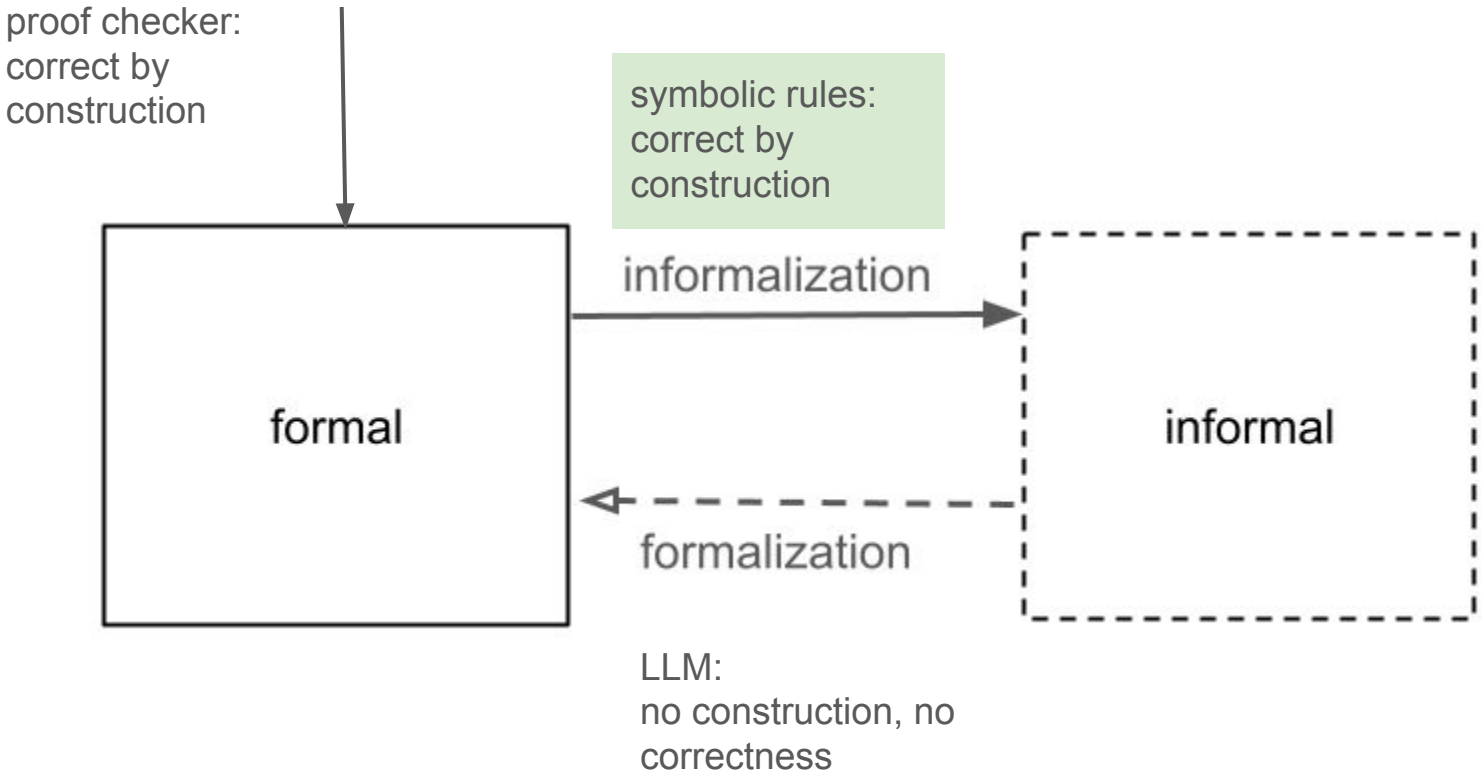
Ideally, you can get the same text back, but now formally checked.



formal language
- precisely defined
- possible to check
- possible to cover completely

informal language
- not precisely defined
- impossible to check
- impossible to cover completely


proof checker:
correct by
construction
formal
informalization
informal
formalization
LLM:
no construction, no
correctness


proof checker:
correct by
construction
symbolic rules:
correct by
construction
informalization
formal
informal
formalization
LLM:
no construction, no
correctness

The project Informath: goals

Semantic completeness: everything expressible in type theory can be translated to Informath natural language.

Semantic soundness: everything expressible in Informath can be translated to type theory.

Linguistic completeness: everything attested in mathematical texts should be eventually covered by Informath.

Linguistic soundness: nothing should be added to Informath unless it is attested in mathematical texts.

The project Informath: the goal in focus here

Semantic completeness: everything expressible in type theory can be translated to Informath natural language.

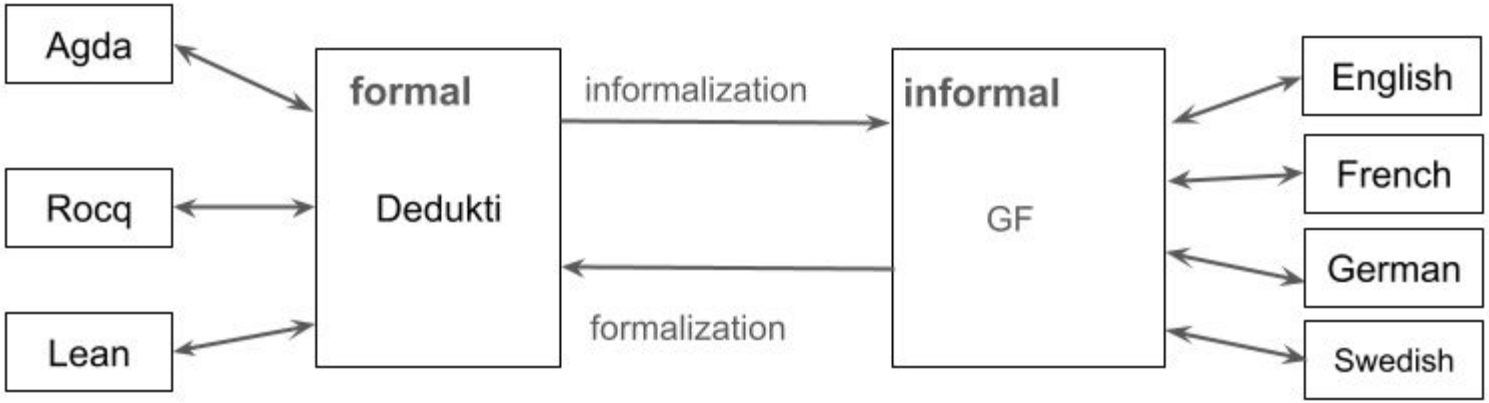
Semantic soundness: everything expressible in Informath can be translated to type theory.

Linguistic completeness: everything attested in mathematical texts should be eventually covered by Informath.

Linguistic soundness: nothing should be added to Informath unless it is attested in mathematical texts.

That will mean lots of translation rules: how can we ever manage to do it?

The project Informath: structure



https://www.grammaticalframework.org/
https://github.com/GrammaticalFramework/informath

Informath statistics

Syntax rules: ~ 200 GF functions, extends to 87,000 BNF rules in English

Lexicon: ~ 4000 items

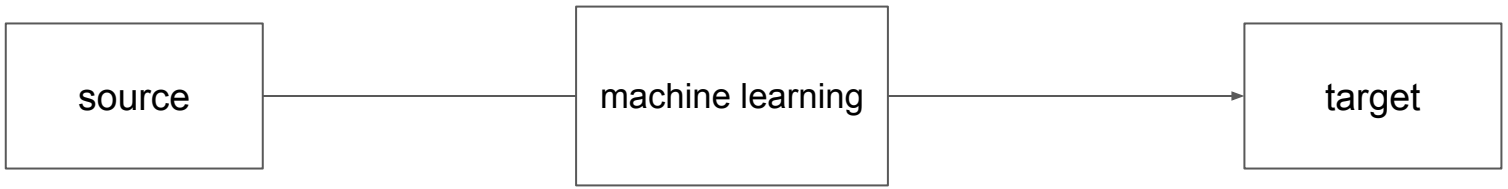
Languages: English, French, German, Swedish

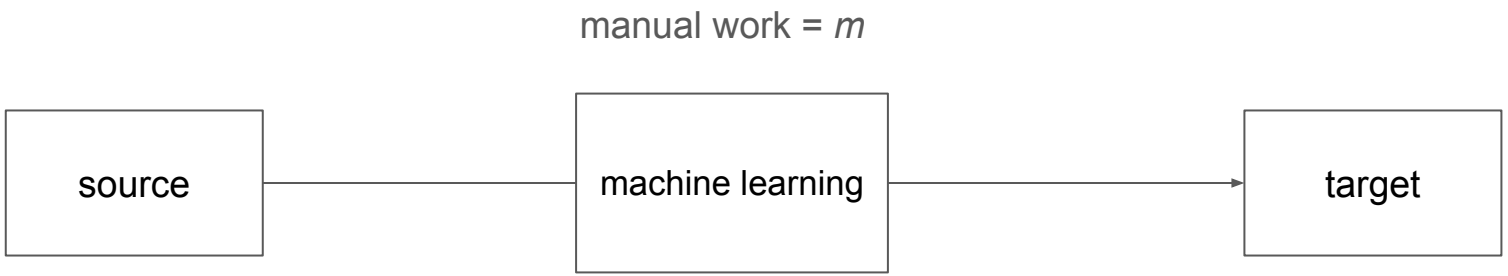
Formalisms: Dedukti; Agda, Lean, Rocq, Megalodon, Mizar

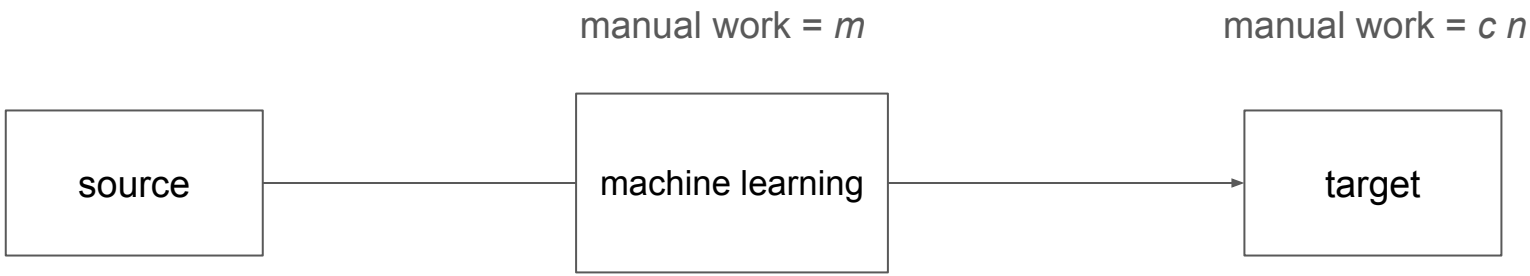
Examples:

- arithmetic, algebra, set theory, topology, a sample of Wiedijk's "100 theorems", homotopy type theory;
- Dedukti dumps from Lean, Matita, Megalodon, Agda

The economy of rules vs. machine learning


source
machine learning
target

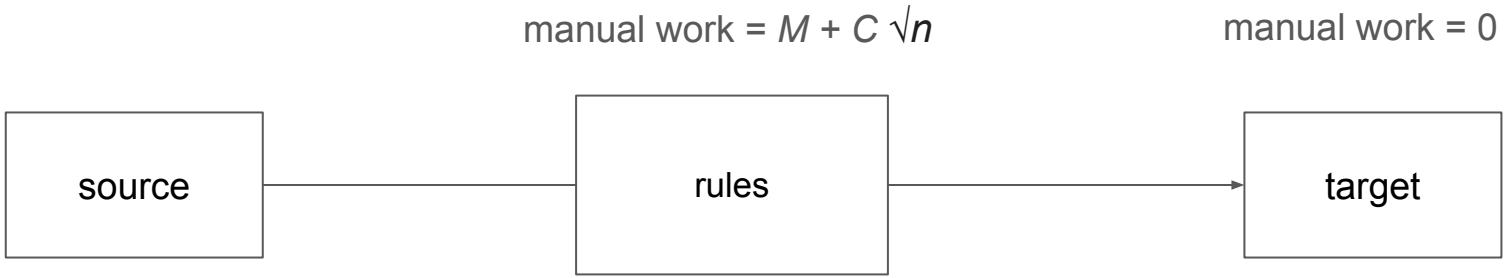

manual work = *m*
source
machine learning
target



total cost = *m* + *c n*

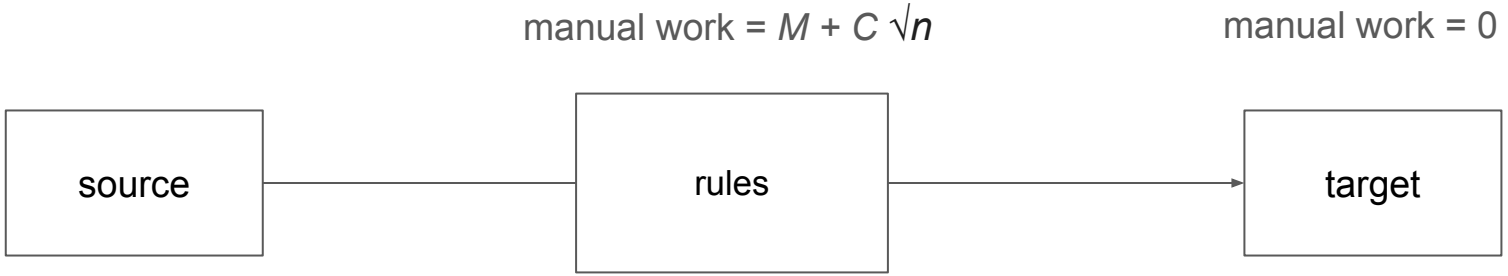
where *n* is the number of units translated


source
rules
target



Total cost = $M + C\sqrt{n}$

- or some other sublinear *f(n)*



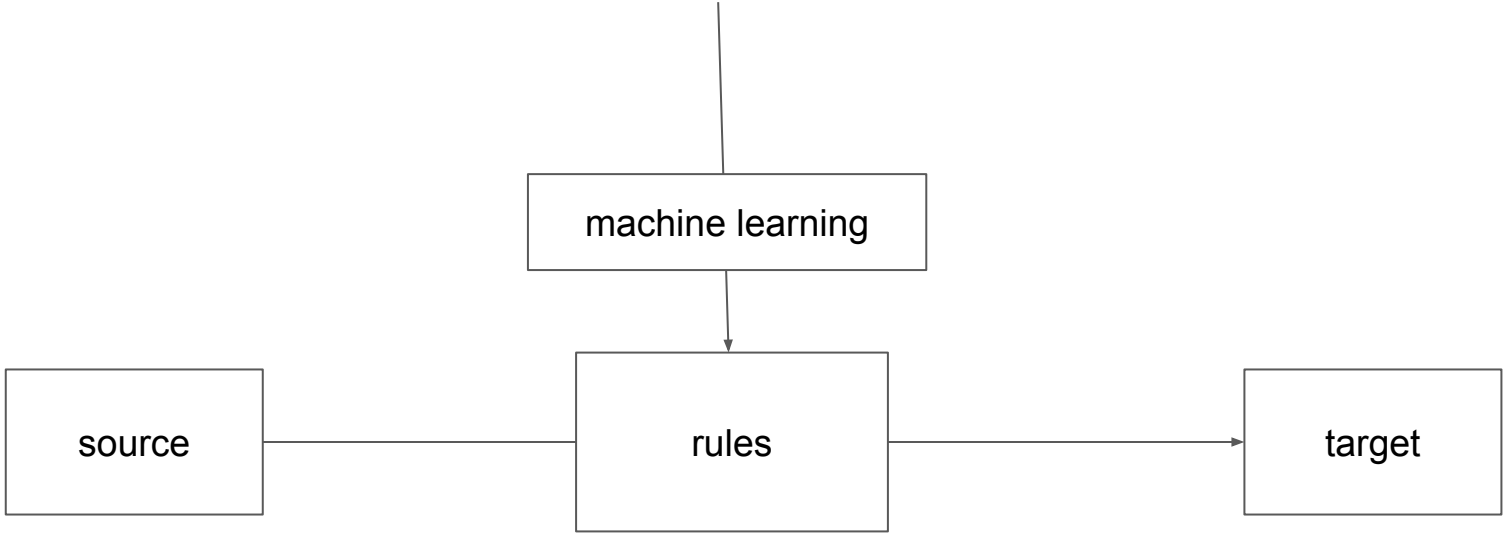
Total cost = $M + C\sqrt{n}$

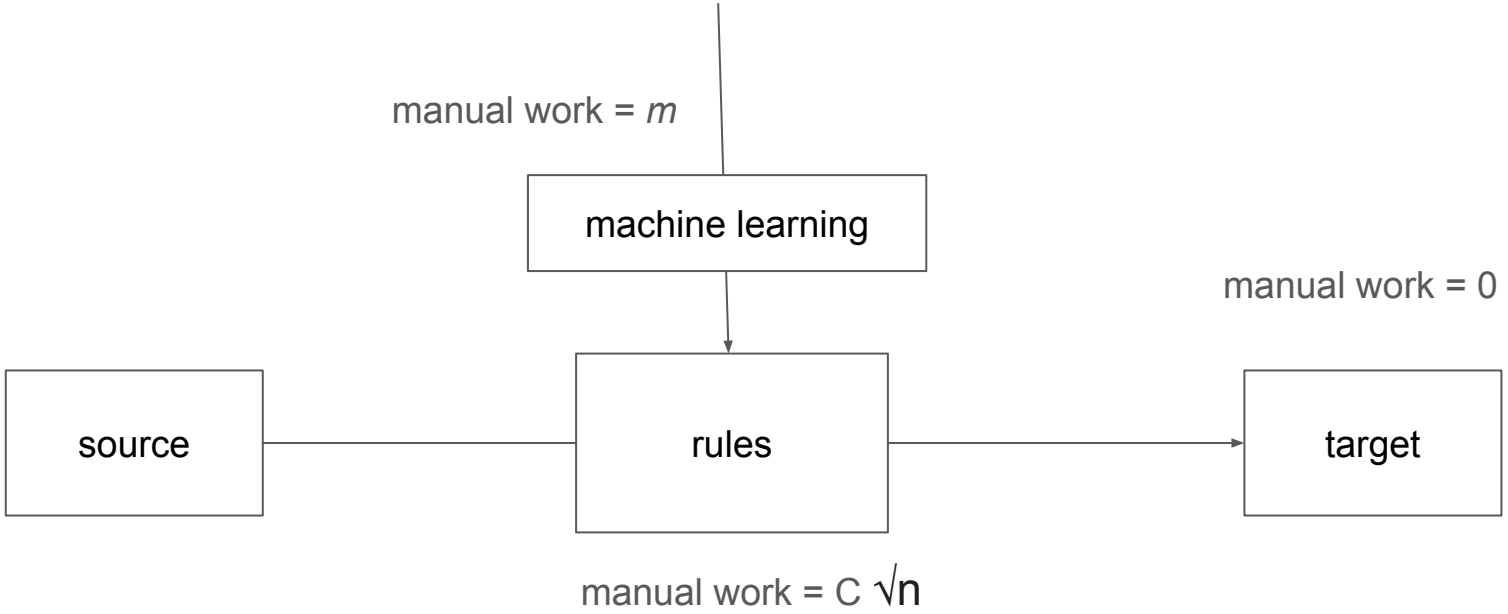
- or some other sublinear $f(n)$

Hence, for large enough n,

$$M + C\sqrt{n} < m + c\,n$$

(Normally, $m \ll M$, $c \ll C$,)


machine learning
source
rules
target



total cost

- machine learning: $m + c\,n$
- manual rules: $M + C\,\sqrt{n}$
- machine-learned rules: $m + C\,\sqrt{n}$

Informath statistics, agent-coded extensions

Syntax rules: ~ 200 GF functions, extends to 87,000 BNF rules in English

Lexicon: ~ 4000 items

Languages: English, French, German, Swedish, Czech, Finnish, Polish

Formalisms: Dedukti; Agda, Lean, Rocq, Megalodon, Mizar, Naproche/Felix

Examples:

- arithmetic, algebra, set theory, topology, a sample of Wiedijk's "100 theorems", homotopy type theory (Rijke 9-15);
- Dedukti dumps from Lean, Matita, Megalodon, Agda
- Godement's Algebra
- Rijke's whole book

Code: Improved proof generation, regression test suite, speed-ups, bug fixes

Over to agent-coded slides…

https://github.com/GrammaticalFramework/informath/blob/main/doc/claude-commits-slides.pdf

Some conclusions: what worked for me

Let LLM generate code, not informal language

- verifiable, type-checkable, or at least mechanically testable
- another argument for strong static checking

Work against a structured code base, not from scratch

- to start, sketch code with holes, bugs, open ends
- replicate analogous code
- use LLM to build and extend regression tests
- iterate by asking LLM to correct its own code

Clearly mark code status

- let LLM create separate pure LLM Git commits
- ask it to mark confidence levels when guessing

Why I changed the title

Key Differences

Feature	Vibe Coding	Agentic Coding / Engineering
Core Mindset	Improvisational and intuition-led; you vibe-check if it feels right.	Disciplined and architect-led; you own the system design and review the logic.
Planning & Specs	Minimal to none; you prompt-and-pray or iterate line-by-line.	Built on upfront written specs, clear constraints, and architectural boundaries.
Agent Autonomy	The AI generates chunks of code in a single chat session, and you test manually.	Autonomous agents plan, execute, run tests, fix bugs, and open pull requests.
Verification	Skimming the UI or output to see if it "looks like it works".	Automated tests, linting, type checks, and rigorous security/risk reviews.
Best Used For	Quick weekend prototypes, throwaway scripts, and early ideation.	Production-grade software, enterprise apps, and scalable codebases.

https://www.google.com/search?q=agentive+vs+vibe&sca_esv=3c71a091e253e848&rlz=1C5CHFA_enSE1068SE10%2068&sxsrf=APpeQnvoiBB5tEk46rqkwITVERsjSjJyoq%3A1792096521259&ei=CTnRapqsD9jEwPAP5Z_GoQs&biw=1310&bih=857&uact=5&oq=agentive+vs+vibe&gs_lp=Egxnd3Mtd2l6LXNlcnAiEGFnZW50aXZlIHZzIHZpYmUyBxAAGIAEGA0yBhAAGBYYHjIGEAAYFhgeMgYQABgWGB4yCBAAGAgYHhgNMggQABgIGB4YDTIIEAAYCBgeGA0yChAAGAgYHhgNGAoyCBAAGAgYHhgNMggQABgIGB4YDUjqLIDLCIj6JHADeAGQAQCYAZYBoAHRC6oBBDEyLjS4AQPIAQD4AQGYAhOgAtcMwgIKEAAYRxjWBBiwA8ICDhAAGOQCGNYEGLAD2AEBwgIXEC4Y3AYYuAYY2gYY2AIYyAMYsAPYAQHCAgQQIxgnwgILEAAYgAQYigUYkQLCAhAQLhiABBiKBRhDGMcBGNEDwgIFEAAYgATCAgoQABiABBiKBRhDwgIKEC4YgAQYiguUYQ8ICCBAAGIAEGMsBwgIKEC4YgAQYywEYCsICBxAAGIAEGArCAgkQABiABBgKGAvCAgwQABiABBgKGAsYtAfCAgYQABgeGA2YAwCIBgGQBg26BgYIARABGAmSBwQxNS40oAfsgAGyBwQxMi40uAe_DMIHCDAuMy4xNS4xyAdagAgB&sclient=gws-wiz-serp

Going up one metalevel

```
> I would like you to create a CLAUDE.md file that enables people to do
autoformalization experiments similar to godement-index and hott-index. The process
should be: read a LaTeX file, formalize its mathematical content in Dedukti, maintain
correctness with dk check, build Informath symbol table with both verbal and symbolic
expressions as close to the LaTeX as possible, maintain its correctness by checking
with RunInformath, create a grammar extension if needed, and present the results in
an doc-index.html file similar to godement-index.html as well as pdf files for
proofs. You can consult the files in ../autoformalize for hints or create skills
corresponding to them, whatever is the best.
```

https://github.com/GrammaticalFramework/informath/blob/main/autoformalize/CLAUDE.md